PROJECT REPORT

Data Cleaning and Transformation Strategy for Road Accident Severity Prediction

Dataset: US Accidents (2016–2023), Sobhan Moosavi et al., Kaggle

Virtual Data Science Apprentice Program — Python Specialist Intern
Week 2 Task: Data Cleaning and Transformation Documentation
Program Duration: 10-Aug-2026 to 07-Sep-2026
Submission Date: 19-Aug-2026

Table of Contents

1. Introduction and Objectives

Building on the Week 1 project proposal, this document defines the data cleaning and transformation strategy for the US Accidents (2016–2023) dataset before it is used to train a road-accident-severity classification model. Raw, real-world data of this scale is never analysis-ready: it contains missing values, duplicate records, inconsistent categorical labels, mismatched data types, and statistical outliers. The purpose of this task is to plan — and demonstrate with worked, reproducible calculations — exactly how each of these issues will be diagnosed and resolved using Python before any modeling begins.

This is a planning and documentation exercise: the calculations below are worked either on the dataset's known, published characteristics (cited where taken from public documentation) or on small illustrative samples that mirror the real column structure, so that the method, formulas, and code are concrete and reproducible rather than only described in the abstract.

1.1 Objectives

- Quantify data quality issues (missingness, duplication, type mismatches, outliers) using explicit formulas rather than general statements.
- Define a column-by-column missing-value strategy with a documented decision rule and a worked numeric example for each rule.
- Specify duplicate-detection logic appropriate to accident records (which are not guaranteed to have a unique natural key).
- Define an outlier-detection and treatment strategy for the key numeric fields, with a fully worked calculation.
- Define the normalization/scaling approach with a worked before-and-after numeric example.
- Specify the feature-engineering steps needed to convert raw timestamp and categorical fields into model-ready features.

2. Dataset Recap and Data Quality Overview

The dataset (as scoped in Week 1) contains approximately 7,728,394 records and 46 columns spanning identifiers, location fields, time fields, weather fields, road-feature boolean flags, and the target column, Severity (an ordinal scale from 1 to 4). Columns fall into four broad dtype groups after an initial pandas load:

Dtype group	Approx. column count	Examples	Cleaning concern
float64	13	Temperature(F), Visibility(mi), Wind_Speed(mph), Precipitation(in), Start_Lat/Lng	Missing values, outliers, unit consistency
object (string)	20	Weather_Condition, City, State, Street, Wind_Direction	Inconsistent labels, free-text noise, high cardinality

Dtype group	Approx. column count	Examples	Cleaning concern
bool	13	Junction, Crossing, Traffic_Signal, Give_Way, Bump	Rarely missing; mainly type-casting checks
int64 / datetime	1 + timestamp cols	Severity, Start_Time, End_Time	Datetime parsing, timezone consistency

A known, documented data-quality characteristic of this dataset is useful as a concrete starting reference point: independent published EDA of the US Accidents dataset reports that the End_Lat and End_Lng columns are missing in roughly 44% of records — by far the highest missing rate in the dataset — because early collection periods did not consistently capture an accident's end coordinates.

Worked calculation — missing-value rate

```
missing_rate(column) = count_missing(column) / total_rows
For End_Lat / End_Lng: missing_rate ≈ 0.44  (44%)
count_missing ≈ 0.44 × 7,728,394 ≈ 3,400,493 rows
Decision rule applied in Section 3.3: missing_rate > 0.40 → drop column
```

This single documented figure anchors the missing-value policy defined in Section 3: any column crossing the same 40% threshold is dropped rather than imputed, because imputing beyond that level would manufacture more than a third of a feature's values.

3. Handling Missing Values

3.1 Diagnostic step

The first Python step is a per-column missingness audit, run before any other cleaning step:

Pseudo-code — missingness audit

```
import pandas as pd
missing = df.isna().sum()
missing_pct = (missing / len(df)) * 100
report = pd.DataFrame({'missing_count': missing,
                       'missing_pct': missing_pct})
                 .sort_values('missing_pct', ascending=False)
```

3.2 Categorizing columns by missingness

Based on the audit output, each column is assigned to one of four bands, each with a different treatment rule:

Missing-rate band	Rule	Rationale	Illustrative example
0% – 5%	Impute (median for numeric, mode for categorical)	Low information loss from imputation	Temperature(F) missing on 3.1% of rows → fill with column median
5% – 40%	Impute with a flag column added	Preserves signal that a value was originally absent	Wind_Chill(F) missing on ~18% of rows → impute + add Wind_Chill_was_missing
> 40%	Drop the column	Imputing a majority-missing column adds more noise than signal	End_Lat / End_Lng at ~44% missing → dropped (not needed for a non-geospatial baseline model)
Target column (Severity)	Drop the row	A model cannot be trained or evaluated on a record with no label	Any row with a null Severity value is removed entirely

3.3 Worked example — median imputation

To make the imputation rule concrete rather than descriptive, the table below is a small illustrative sample of ten Temperature(F) values, mirroring the real column's structure (continuous, occasional nulls):

Sample values (°F)

```
72, 68, NaN, 75, 71, NaN, 69, 74, 70, 73
```

Worked calculation — median imputation

```
Non-null values sorted: 68, 69, 70, 71, 72, 73, 74, 75  (n = 8)
median = average of 4th and 5th values = (71 + 72) / 2 = 71.5
Both NaN entries are replaced with 71.5
Resulting mean shifts from 71.5 (n=8) to 71.5 (n=10) — unchanged here because
the median equals the pre-imputation mean in this sample, which is the
intended effect: imputation should not distort central tendency.
```

The same logic is applied with the categorical mode for object columns, for example Wind_Direction, where the most frequent category (commonly ‘CALM’ or ‘VAR’ depending on region) replaces missing entries.

3.4 Python implementation outline

Pseudo-code — tiered imputation

```
for col in numeric_cols:
```

```
    pct = missing_pct[col]
    if pct > 40: df.drop(columns=[col], inplace=True); continue
    if pct > 5:  df[col+'_was_missing'] = df[col].isna().astype(int)
    df[col] = df[col].fillna(df[col].median())

for col in categorical_cols:
    if missing_pct[col] > 40: df.drop(columns=[col], inplace=True); continue
    df[col] = df[col].fillna(df[col].mode()[0])

df = df.dropna(subset=['Severity'])
```

4. Handling Duplicate Records

Accident records in this dataset do not carry a single guaranteed natural key across all source feeds, so duplicate detection uses a composite key rather than a plain row-level check.

- Exact duplicates: full-row duplicates are dropped with `df.drop_duplicates()` as a first pass.
- Near-duplicates (composite key): records sharing the same `Start_Time, Start_Lat, Start_Lng` (rounded to 4 decimal places, ≈ 11 m precision) are treated as the same physical accident reported twice by different traffic APIs.

Worked calculation — composite-key duplicate rate

```
key = round(Start_Lat, 4), round(Start_Lng, 4), Start_Time
Illustrative sample: 10,000 rows -> 9,830 unique keys
duplicate_rate = 1 - (unique_keys / total_rows)
duplicate_rate = 1 - (9,830 / 10,000) = 0.017  (1.7%)
Action: keep the first occurrence per key, drop the remaining 170 rows
```

The 1.7% figure above is an illustrative sample calculation demonstrating the formula and code path; the actual rate will be measured directly once the full dataset is loaded in Week 2 execution, and the same formula will be re-run to report the true value.

5. Handling Outliers

Outlier treatment is scoped to continuous numeric columns most likely to contain sensor or reporting errors: Temperature(F), Wind_Speed(mph), Visibility(mi), and Precipitation(in). The Interquartile Range (IQR) method is used because it is robust to the heavy skew already known to exist in weather data (for example, Precipitation(in) is zero for the large majority of records and only non-zero during active rain/snow events).

5.1 Worked example — IQR method on Wind_Speed(mph)

Using an illustrative 12-value sample consistent with the real column's typical range and units:

Sample values (mph)

```
5, 7, 8, 9, 10, 10, 11, 12, 13, 15, 16, 92
```

Worked calculation — IQR bounds

```
Q1 (25th percentile) = 8.5
Q3 (75th percentile) = 14.0
IQR = Q3 - Q1 = 14.0 - 8.5 = 5.5
Lower bound = Q1 - 1.5 * IQR = 8.5 - 8.25 = 0.25
Upper bound = Q3 + 1.5 * IQR = 14.0 + 8.25 = 22.25
92 mph > 22.25 -> flagged as an outlier and capped/removed
```

In practice, values above the upper bound are winsorized (capped at the bound) rather than dropped outright for weather fields, since a genuinely high wind-speed reading during a storm is meaningful signal for accident severity rather than a data-entry error; a secondary domain rule (e.g., Wind_Speed(mph) > 150 is treated as a sensor/unit error and set to null for re-imputation) is applied before the statistical rule.

5.2 Python implementation outline

Pseudo-code — IQR capping

```
def cap_outliers_iqr(series, k=1.5):
    q1, q3 = series.quantile(0.25), series.quantile(0.75)
    iqr = q3 - q1
    lower, upper = q1 - k*iqr, q3 + k*iqr
    return series.clip(lower=lower, upper=upper)

for col in ['Wind_Speed(mph)', 'Visibility(mi)', 'Temperature(F)']:
    df[col] = cap_outliers_iqr(df[col])
```

6. Data Type Standardization

- Start_Time and End_Time are parsed with `pd.to_datetime()`, with a validation check that fewer than 0.1% of rows fail to parse before proceeding (rows that fail are logged, not silently dropped).
- Boolean flag columns (Junction, Crossing, Traffic_Signal, etc.) are cast explicitly to `bool` to avoid them being read as object/string 'True'/'False' text.
- Free-text Weather_Condition values are standardized to a smaller controlled vocabulary — for example, 'Light Rain', 'Light Rain / Windy', and 'Light Rain with Thunder' are consolidated under a base category `Rain` plus separate boolean modifier flags `is_windy` and `has_thunder`, reducing category count and cardinality-driven overfitting risk.

Worked calculation — cardinality reduction

```
Illustrative: Weather_Condition has 90 raw unique string values
```

```
After consolidation to base condition (Rain, Snow, Fog, Clear, ...) + 2 modifier flags:
unique_base_categories ≈ 12
cardinality_reduction = 1 - (12 / 90) = 0.867  (≈87% fewer categories)
Effect: one-hot encoding in Week 4 produces 12 columns instead of 90
```

7. Feature Scaling and Normalization

Numeric features on different scales (e.g., Temperature(F) in the tens to hundreds, Precipitation(in) typically under 1.0) will be standardized before use in distance-based or gradient-based models. Two approaches are planned and compared:

Method	Formula	When used
Min-Max scaling	x' = (x - min) / (max - min)	Neural-network-style models where bounded [0,1] inputs are preferred
Z-score standardization	x' = (x - mean) / std	Logistic regression and other models sensitive to feature variance

7.1 Worked example — z-score standardization

Using the same illustrative Temperature(F) sample from Section 3.3 (after imputation: 72, 68, 71.5, 75, 71, 71.5, 69, 74, 70, 73):

Worked calculation — mean, std, and z-score

```
mean = (72+68+71.5+75+71+71.5+69+74+70+73) / 10 = 71.5
variance = sum((x - mean)^2) / n = 5.65
std = sqrt(5.65) ≈ 2.377
z-score for x = 75:  z = (75 - 71.5) / 2.377 ≈ 1.473
z-score for x = 68:  z = (68 - 71.5) / 2.377 ≈ -1.473
```

The symmetry of the two results above (±1.473) is an expected sanity check: 75 and 68 sit equally far on either side of the mean, so their z-scores should be equal in magnitude and opposite in sign — a quick way to catch a formula or sign error during implementation.

8. Feature Engineering from Cleaned Data

- Time-based features: Hour, DayOfWeek, Month, and IsWeekend are extracted from Start_Time; Duration_Minutes is computed as `(End_Time - Start_Time).dt.total_seconds() / 60`.

- Geospatial simplification: Start_Lat/Start_Lng are retained; State and City are kept as categorical features rather than raw coordinates for the baseline model, per the Week 1 scope decision to exclude geospatial modeling.
- Encoding: low-cardinality categoricals (State, base Weather_Condition) use one-hot encoding; higher-cardinality categoricals (City, Street) use frequency or target encoding to avoid a very wide sparse matrix.

Worked calculation — Duration_Minutes

```
Start_Time = 2023-01-05 14:02:00
End_Time   = 2023-01-05 14:47:00
Duration_Minutes = (End_Time - Start_Time) in seconds / 60
                 = 2700 seconds / 60 = 45.0 minutes
```

9. Cleaning and Transformation Workflow Diagram

Raw CSV Load → *Missingness Audit* → *Column Drop / Impute Decision* → *Duplicate Detection* → *Outlier Capping (IQR)* → *Type Standardization* → *Category Consolidation* → *Scaling / Standardization* → *Feature Engineering* → *Cleaned, Model-Ready DataFrame*

Each arrow is a validation checkpoint (Section 10): the pipeline does not proceed to the next stage until the output of the current stage passes its stated check, and the row/column count and null count are logged at every stage for auditability.

10. Validation Checkpoints

Stage	Validation check	Pass criterion
Missingness audit	Per-column missing % report generated	Report covers all 46 columns
Impute / drop	No nulls remain in retained columns (except intentional flag logic)	df.isna().sum().sum() == 0 for numeric/categorical features
Duplicates	Composite-key duplicate rate re-measured post-drop	Duplicate rate == 0% on final dataset
Outliers	Values outside IQR bounds capped, not silently dropped	Row count unchanged after capping step
Types	dtypes match target schema (datetime64, bool, category)	df.dtypes matches documented schema
Scaling	Mean ≈ 0, std ≈ 1 on standardized columns (sample check)	Within ±0.01 tolerance on a validation subsample

11. Anticipated Challenges and Mitigations

Challenge	Mitigation
Dataset size (7.7M rows) makes iterative testing on a laptop slow	Develop and validate the pipeline on a stratified 2% sample first, then run on the full dataset
IQR bounds computed on a skewed weather column may over-flag legitimate extreme weather	Apply a domain sanity ceiling (Section 5.1) before the statistical rule, and review flagged rows in aggregate rather than deleting automatically
Category consolidation for Weather_Condition is a manual mapping and could miss rare strings	Log any string not matched by the mapping to an 'Other/Unmapped' bucket and review the size of that bucket before finalizing
Imputing the target-adjacent weather features could leak information if done using the full dataset including test rows	Fit all imputation statistics (median, mode) on the training split only, then apply to validation/test splits

12. Conclusion

This document translates the general cleaning objectives from the Week 1 proposal into a concrete, threshold-driven, and numerically worked plan: a documented 44% missing-rate reference point anchors the drop-vs-impute rule; a worked median-imputation example, a worked IQR outlier calculation, a worked z-score standardization calculation, and a worked duration-feature calculation each show the exact formula applied to representative values; and every pipeline stage carries an explicit, auditable validation check. This provides a directly implementable specification for the Week 2 Python data-cleaning script and a clean, well-understood input for the Week 3 exploratory data analysis phase.